

Python Assessment – Questions

Problem Statement

You need to implement a **Word Frequency Counter** that takes a sentence as input and returns a dictionary where:

- Keys → Words
- Values → Word frequencies

The implementation should handle punctuation and case insensitivity.

Class Declaration

```
class WordFrequencyCounter:
    No need for __init__ method
    If used, follow recommendations:
    def __init__(self):
        pass
```

Operations

1. Clean and Tokenize Input

Removes punctuation, converts the sentence to lowercase, and tokenizes words.

Function Prototype:

```
def preprocess_text(self, text: str) -> list:
```

Example Input:

```
preprocess_text("Hello, hello! How are you?")
```

Expected Output:

```
['hello', 'hello', 'how', 'are', 'you']
```

Implementation Flow:

- Convert the input text to lowercase.
- Remove punctuation using `re.sub()`.
- Split text into words using `.split()`.

2. Compute Word Frequency

Counts occurrences of each word in a dictionary.

Function Prototype:

```
def compute_word_frequency(self, words: list) -> dict:
```

Example Input:

```
compute_word_frequency(['hello', 'hello', 'how', 'are', 'you'])
```

Expected Output:

```
{'hello': 2, 'how': 1, 'are': 1, 'you': 1}
```

Implementation Flow:

- Declare an empty dictionary.
- Iterate through the list using a for loop.
- Use `dictionary.get(word, 0)` to count occurrences.
- Return a dictionary where keys = words and values = frequencies.

3. Get Most Frequent Word

Returns the most frequent word and its count.

Function Prototype:

```
def get_most_frequent_word(self, freq_dict: dict) -> tuple:
```

Example Input:

```
get_most_frequent_word({'hello': 2, 'how': 1, 'are': 1, 'you': 1})
```

Expected Output:

```
('hello', 2)
```

Implementation Flow:

- If nothing in dictionary, return None.
- Iterate through the dictionary to find the word with the highest count.
- Use max() for highest.
- Use lambda x: x[1] for mapping keys.
- Return the word and its count as a tuple.

4. Filter Words by Minimum Frequency

Returns words that appear at least a specified number of times.

Function Prototype:

```
def filter_words_by_frequency(self, freq_dict: dict, n: int) -> dict:
```

Implementation Flow:

- If count > n, iterate over the dictionary using a for loop and include only words that meet or exceed the threshold.
- Else return an empty dictionary.

Problem Statement

An organization tracks daily attendance logs for its employees. Each log entry includes the employee ID, department, date, and attendance status (Present/Absent/Leave). Your task is to perform analysis over this attendance data to identify attendance rates, patterns, and departmental summaries.

Class Declaration

```
class AttendanceAnalyzer:
    No need to use init method
    If needed, follow recommendations:
    def __init__(self):
        pass
```

Operations

1. Create Attendance DataFrame

Converts raw attendance logs into a structured Pandas DataFrame.

Function Prototype:

```
def create_attendance_df(self, data: list) -> pd.DataFrame:
```

Example Input:

```
create_attendance_df([
    [201, "Sales", "2024-06-01", "Present"],
    [202, "HR", "2024-06-01", "Absent"],
    [201, "Sales", "2024-06-02", "Leave"]
])
```

Expected Output:

	EmployeeID	Department	Date	Attendance
0	201	Sales	2024-06-01	Present
1	202	HR	2024-06-01	Absent
2	201	Sales	2024-06-02	Leave

Implementation Flow:

- Create a DataFrame using columns: ["EmployeeID", "Department", "Date", "Attendance"].
- Ensure Date remains as a string (do not convert to datetime).

2. Calculate Monthly Attendance Rate per Employee

Returns a DataFrame showing percentage of days present per employee per month.

Function Prototype:

```
def compute_monthly_attendance_rate(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input: Assuming records from June 2024

Expected Output:

	EmployeeID	Month	Attendance	Rate
0	201	2024-06		50.0
1	202	2024-06		0.0

Implementation Flow:

- Extract "Year-Month" from Date using `df["Date"].str[:7]`. This will remove day and keep rest intact.
- Group by EmployeeID and Month, count and total days.
- Create a new df by applying filter where Attendance = Present.
- Merge original and Present df.
- Compute Attendance Rate = $(\text{Present} / \text{Total}) * 100$.

3. Add Absence Flag Column

Adds a column "IsAbsent" with 1 if status is "Absent", else 0.

Function Prototype:

```
def add_absence_flag(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
add_absence_flag(df)
```

Expected Output:

	EmployeeID	Department	Date	Attendance	IsAbsent
0	201	Sales	2024-06-01	Present	0
1	202	HR	2024-06-01	Absent	1
2	201	Sales	2024-06-02	Leave	0

Implementation Flow:

- Use `df["Attendance"] == "Absent"` and convert boolean to int.

4. Filter Employees with High Absenteeism

Returns a DataFrame of employees with more than n absences.

Function Prototype:

```
def high_absentees(self, df: pd.DataFrame, threshold: int) -> pd.DataFrame:
```

Example Input:

```
high_absentees(df, 2)
```

Expected Output:

	EmployeeID	Absence Count
0	202	3
1	203	4

Implementation Flow:

- Filter rows where Attendance is "Absent".
- Group by EmployeeID and count absences.
- Return only those with count > threshold.

5. Department Attendance Summary

Returns counts of each attendance type per department.

Function Prototype:

```
def department_attendance_summary(self, df: pd.DataFrame) -> pd.DataFrame:
```

Example Input:

```
department_attendance_summary(df)
```

Expected Output:

Department	Present	Absent	Leave
------------	---------	--------	-------

0	HR	2	3	1
1	Sales	3	1	2

Implementation Flow:

- Implement crosstab using "Department" and "Attendance" in the input DataFrame.
- Set column name to None.
- Use df.reindex with column argument as [present, absent, leave].
- Replace null values with 0 using fill_value=0.
- Reset index and return the resultant DataFrame.